Computer Networks

Fall 2024 Semester

Course Code: CS – 3001

Offered to BCS22 – 5K, BDS21 – 7C

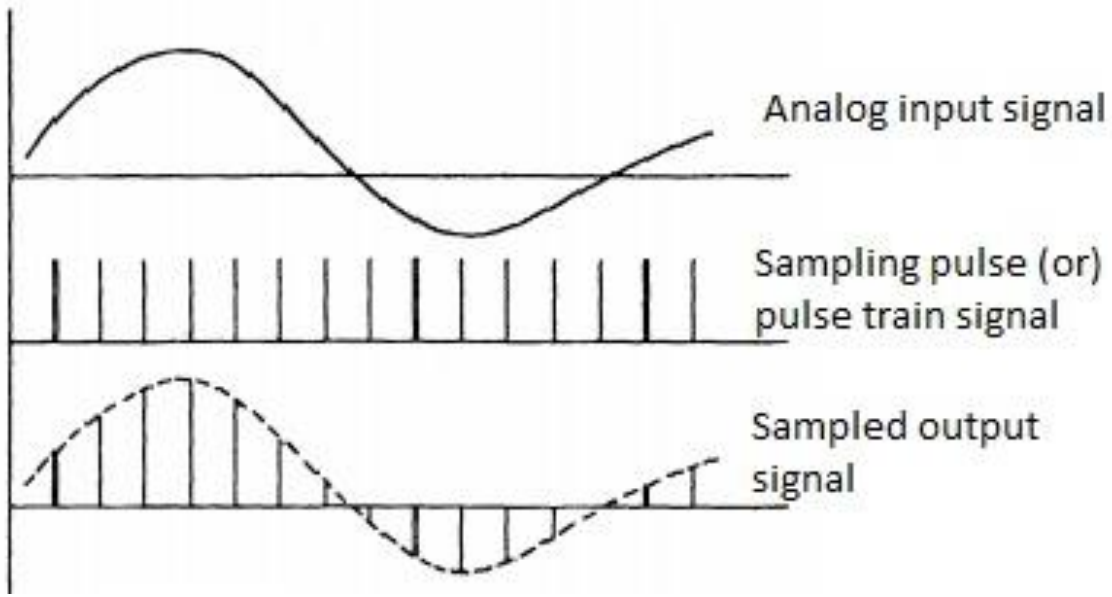
Course Instructor: Aftab Alam

Lecture 5

September 03, 2024 (Tuesday)

Nyquist Theorem

- The Nyquist theorem is a cornerstone of DSP.
- It ensures that continuous signals can be accurately represented in a discrete form as long as the sampling rate is at least twice the maximum frequency present in the signal.



Maximum Data Rate

- During digital transmission, noise over the communication path may affect the digital signal; so we can have limited number of bits per sample.
- Maximum data rate (in bps) (also called **Channel Capacity** or **Nyquist Bandwidth**) is given by the formula

$$C = 2B \log_2 M$$

where

B is the bandwidth of the channel (in Hz)

and

M is the number of voltage levels.

Example

- Consider a voice channel to transmit digital data via MODEM. Assuming a bandwidth of 3100 Hz, what will be the maximum data rate if
  a). If we have 2 voltage levels for binary data
  b). If we have 8 voltage levels for binary data

$$C = 2B \log_2 M$$

a). Maximum data rate = (2)(3100)(1) = 6.2 kbps

b). Maximum data rate = (2)(3100)(3) = 18.6 kbps

Shannon Capacity Formula

- For a given level of noise, greater signal strength would improve the ability to receive data correctly in the presence of noise.
- According to Shannon, the maximum data rate obeys the equation

$$C = B \log_2 (1+SNR)$$

where

B is the bandwidth of the channel (in Hz)

and

SNR (Signal-to-Noise Ratio) = Signal Power/Noise Power

Example

- Suppose that the spectrum of channel is between 3 and 4 MHz, and SNR = 24 dB.

a) Find the maximum data rate that can be achieved?

b) How many signal levels can be accommodated?

Given

Bandwidth = 4 – 3 = 1 MHz

SNR (in dB) = 24 dB = 10 $\log_{10}$ (SNR) ➔ SNR = $10^{2.4}$ = 251.18

Now

C = B $\log_2$ (1+SNR) = (10^6) $\log_2$ (1+251.18) = 8 Mbps

Since

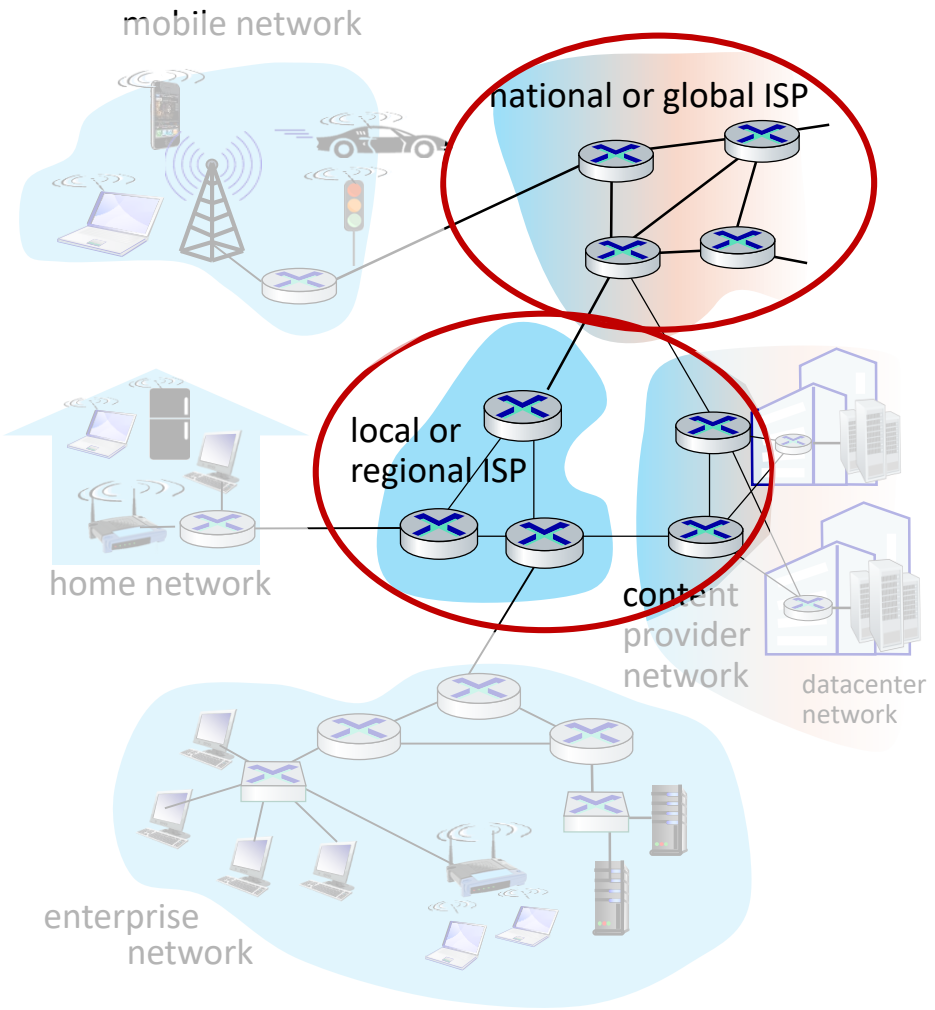
C = 2B log M ➔ 8 x 10^6 = 2 x 10^6 x $\log_2$ M ➔ M = 16

Chapter 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models
- History

The network core

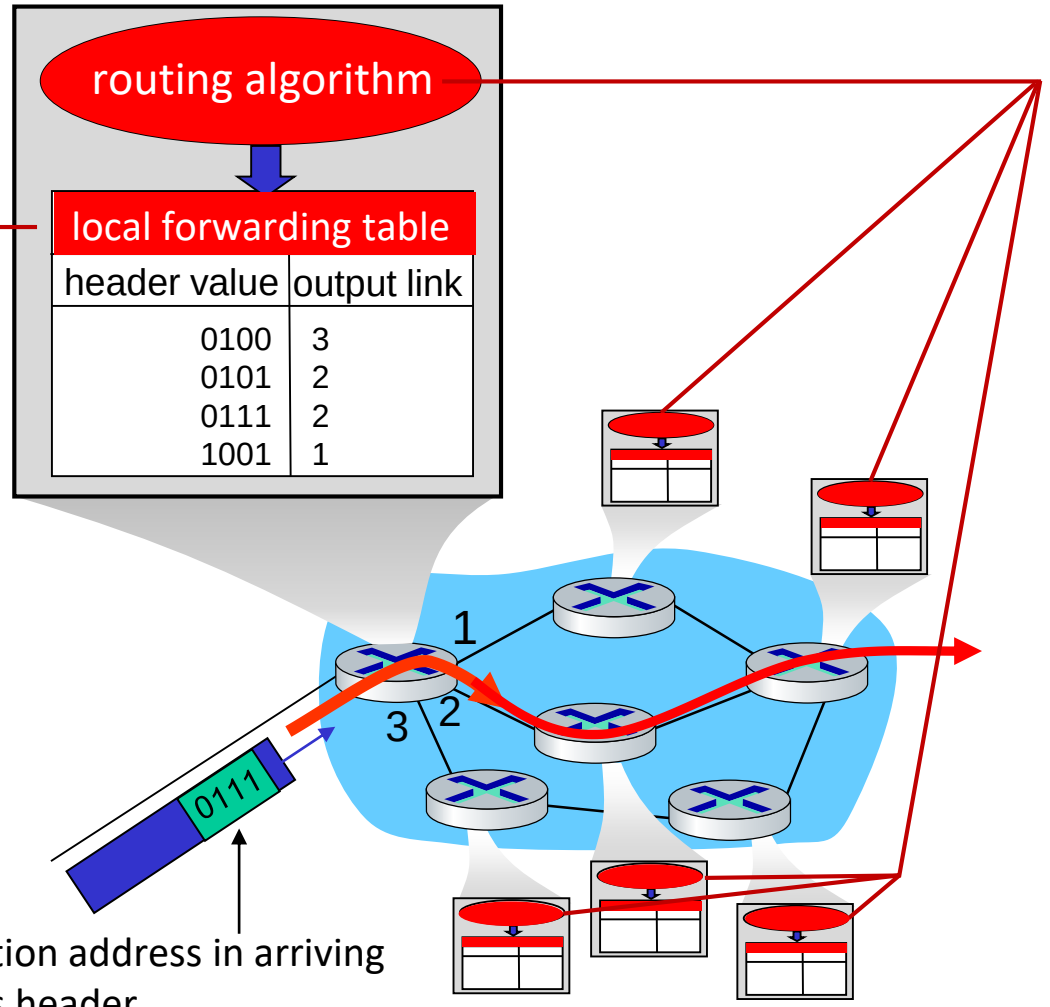
- mesh of interconnected routers
- packet-switching: hosts break application-layer messages into *packets*
  - network forwards packets from one router to the next, across links on path from source to destination



Two key network-core functions

Forwarding:

- aka "switching"
- *local* action: move arriving packets from router's input link to appropriate router output link



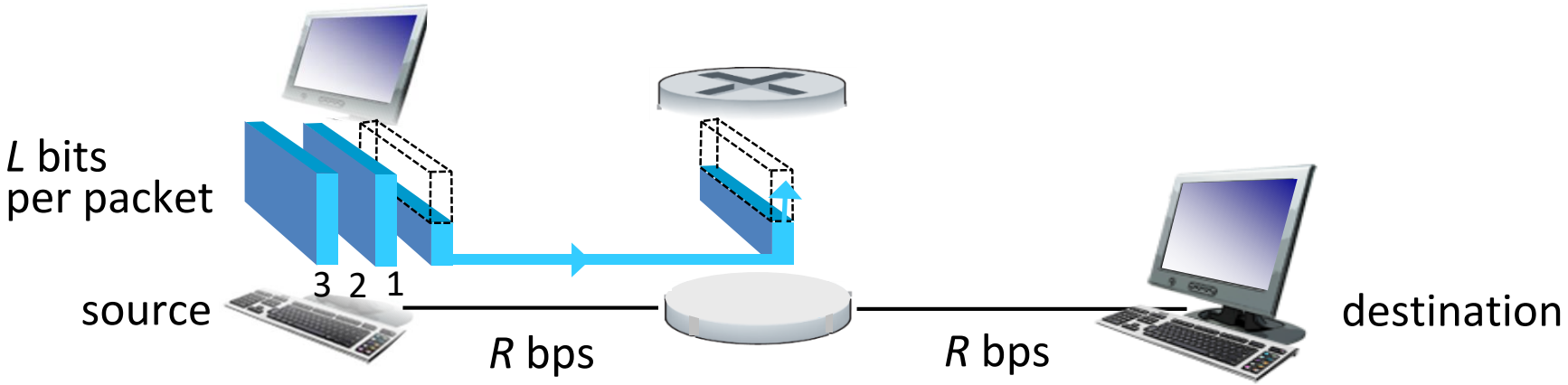
Routing:

- *global* action: determine source-destination paths taken by packets
- routing algorithms

routing
45 h
3,053 miles
48 h
3,309 miles
Northampton
San Jose
United States
Victoria
Seattle
WASHINGTON
Portland
OREGON
IDAHO
MONTANA
WYOMING
NORTH DAKOTA
SOUTH DAKOTA
MINNESOTA
Minneapolis
WISCONSIN
MICHIGAN
Toronto
Detroit
Chicago
IOWA
Quebec City
Montreal
Ottawa
MAINE
VERMONT
NEW HAMPSHIRE
CT
RI
New York
PENNSYLVANIA
Philadelphia
NJ
MARYLAND
DE
WEST VIRGINIA
VIRGINIA
OHIO
INDIANA
Indianapolis
ILLINOIS
KENTUCKY
Nashville
TENNESSEE
NORTH CAROLINA
Charlotte
SOUTH CAROLINA
Atlanta
GEORGIA
ALABAMA
MISSISSIPPI
LOUISIANA
New Orleans
Jacksonville
Orlando
Tampa
FLORIDA
ARKANSAS
MISSOURI
Kansas City
OKLAHOMA
Dallas
TEXAS
Austin
Houston
San Antonio
COAHUILA
CHIHUAHUA
El Paso
NEW MEXICO
COLORADO
UTAH
NEVADA
Las Vegas
ARIZONA
Phoenix
Tucson
SONORA
BAJA CALIFORNIA
CALIFORNIA
Los Angeles
San Diego
Sacramento
San Francisco

forwarding
forwarding
45 h
3,053 miles
48 h
3,309 miles
Victoria
Seattle
WASHINGTON
Portland
NORTH DAKOTA
SOUTH DAKOTA
MINNESOTA
Minneapolis
WISCONSIN
MICHIGAN
Toronto
Ottawa
Montreal
Quebec City
MAINE
VERMONT
NEW HAMPSHIRE
Northampton
Detroit
Chicago
WYOMING
IOWA
ILLINOIS
INDIANA
OHIO
Indianapolis
United States
Kansas City
COLORADO
KANSAS
MISSOURI
KENTUCKY
Nashville
TENNESSEE
San Jose
CALIFORNIA
Las Vegas
OKLAHOMA
ARKANSAS
Los Angeles
ARIZONA
NEW MEXICO
San Diego
Phoenix
Dallas
MISSISSIPPI
ALABAMA
Atlanta
Tucson
El Paso
TEXAS
BAJA CALIFORNIA
SONORA
Austin
Houston
LOUISIANA
New Orleans
CHIHUAHUA
San Antonio
COAHUILA
Tampa
FLORIDA
Cleveland
DOWNTOWN
CENTRAL
OHIO CITY

Packet-switching: store-and-forward

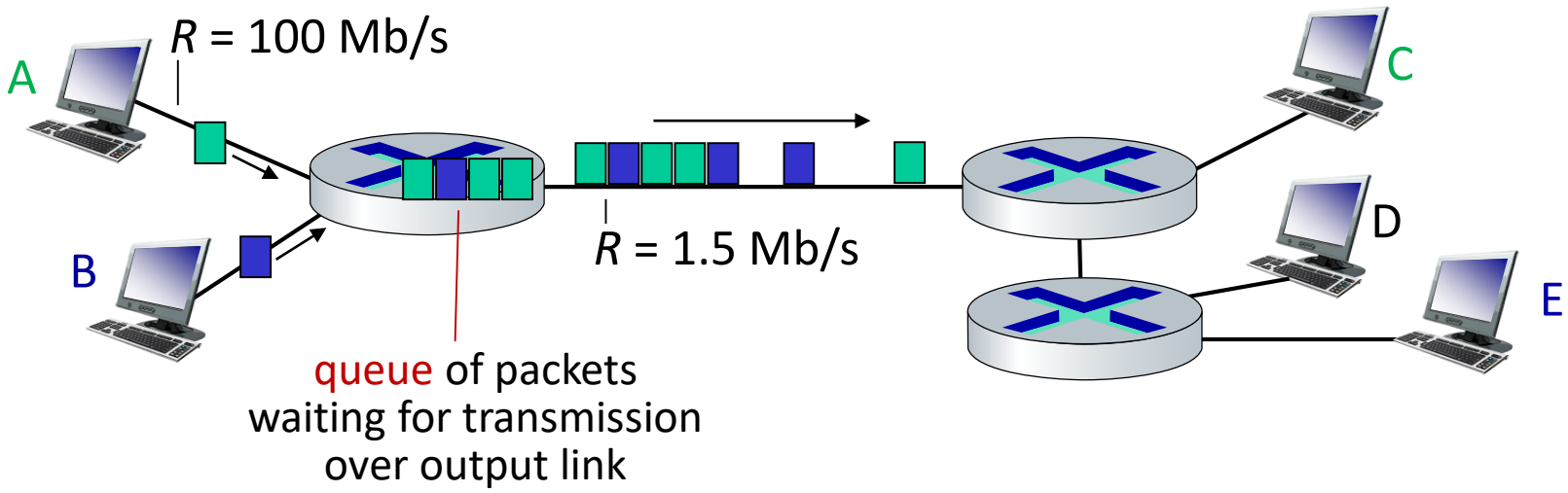


- packet transmission delay: takes L/R seconds to transmit (push out) L-bit packet into link at R bps
- *store and forward: entire* packet must arrive at router before it can be transmitted on next link

One-hop numerical example:

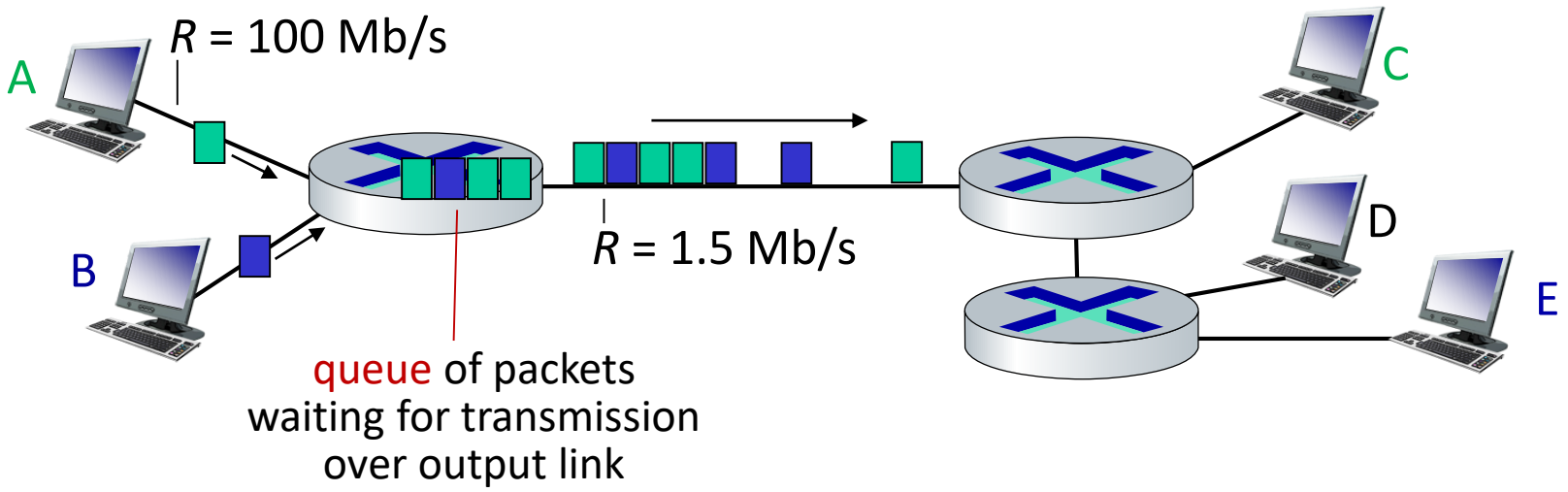
- L = 10 Kbits
- R = 100 Mbps
- one-hop transmission delay = 0.1 msec

Packet-switching: queueing



Queueing occurs when work arrives faster than it can be serviced:

Packet-switching: queueing



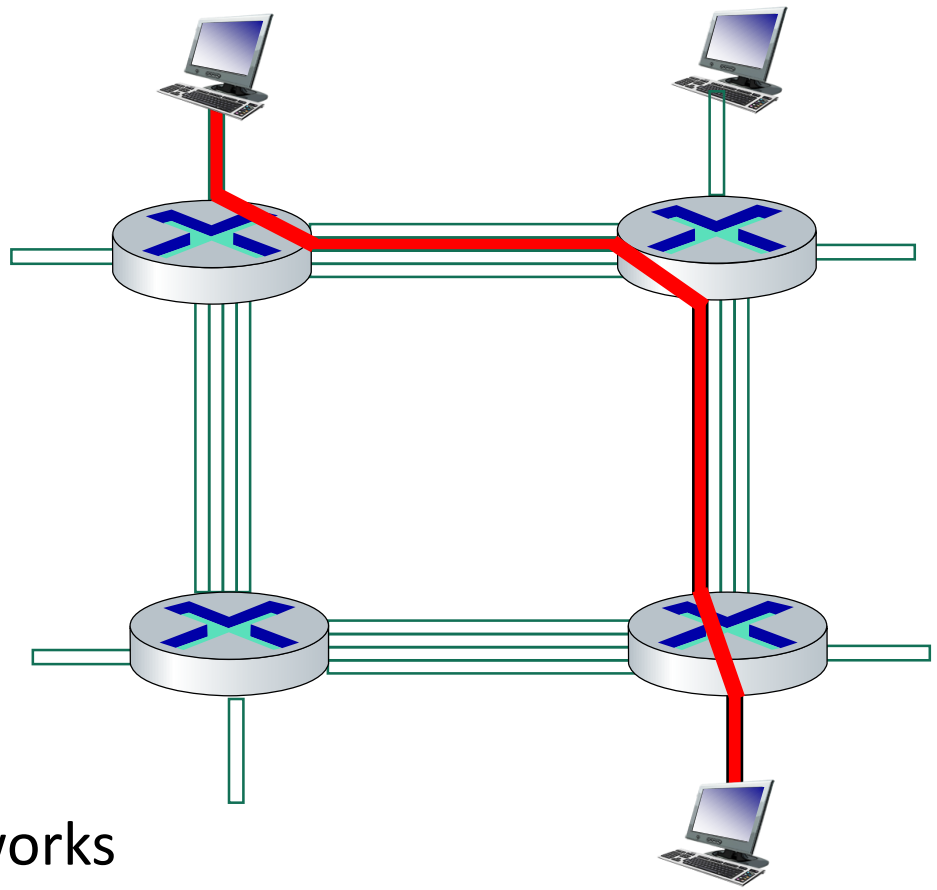
Packet queuing and loss: if arrival rate (in bps) to link exceeds transmission rate (bps) of link for some period of time:

- packets will queue, waiting to be transmitted on output link
- packets can be dropped (lost) if memory (buffer) in router fills up

Alternative to packet switching: circuit switching

end-end resources allocated to, reserved for "call" between source and destination

- in diagram, each link has four circuits.
  - call gets 2nd circuit in top link and 1st circuit in right link.
- dedicated resources: no sharing
  - circuit-like (guaranteed) performance
- circuit segment idle if not used by call (no sharing)
  - commonly used in traditional telephone networks



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive

Circuit Switching vs Packet Switching

Circuit Switching:

- **Connection-Oriented:** Requires a dedicated path for the entire communication session.
- **Example:** Traditional telephone systems.
- **Best For:** Real-time, consistent communication with minimal latency (e.g., voice calls).

Packet Switching:

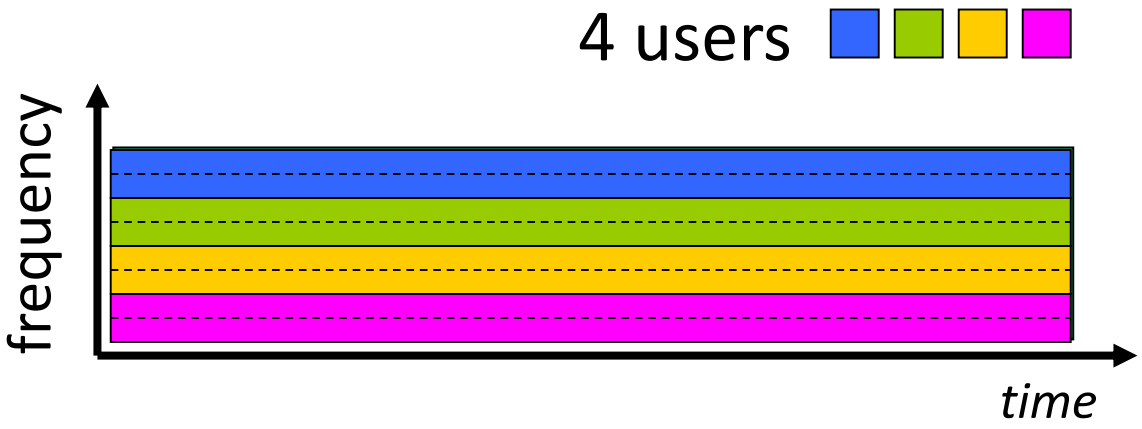
- **Connectionless (Typically):** Data is sent in packets, each of which may take a different route.
- **Example:** Internet data transmission.
- **Best For:** Efficient, scalable communication with potentially variable quality, suitable for data networks.

In modern networks, packet switching is the predominant method, especially for data communication, due to its efficiency and scalability. However, circuit switching is still used in scenarios where consistent quality and minimal latency are crucial, such as in traditional telephony or certain real-time applications.

Circuit switching: FDM and TDM

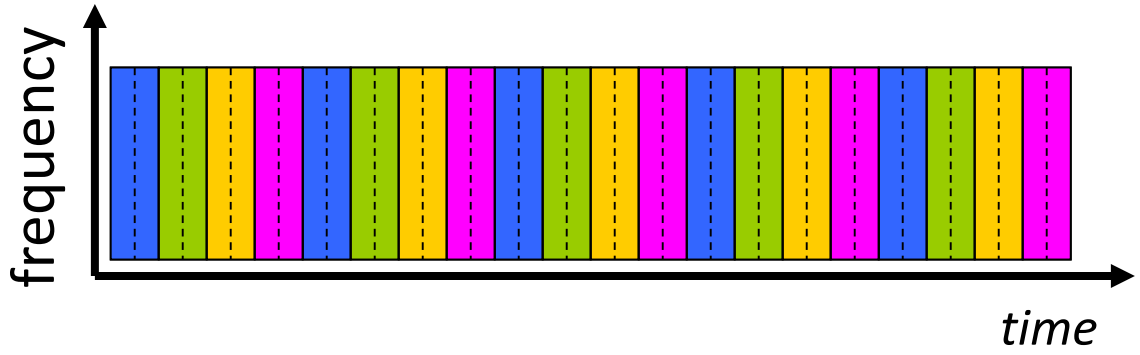
Frequency Division Multiplexing (FDM)

- optical, EM frequencies divided into (narrow) frequency bands
  - each call allocated its own band, can transmit at max rate of that narrow band



Time Division Multiplexing (TDM)

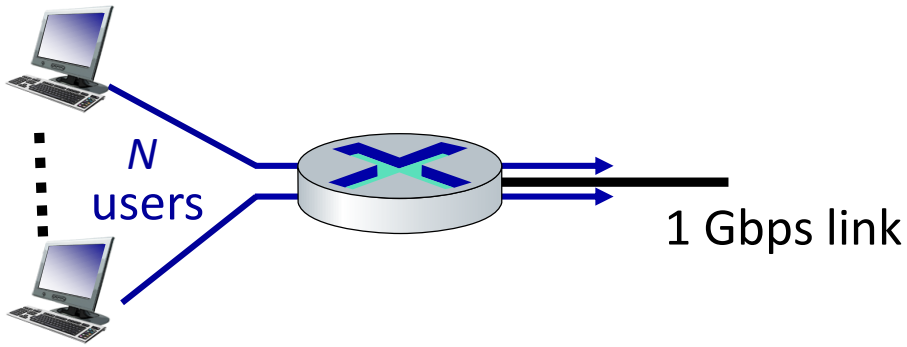
- time divided into slots
- each call allocated periodic slot(s), can transmit at maximum rate of (wider) frequency band (only) during its time slot(s)



Packet switching versus circuit switching

example:

- 1 Gb/s link
- each user:
  - 100 Mb/s when "active"
  - active 10% of time



Q: how many users can use this network under circuit-switching and packet switching?

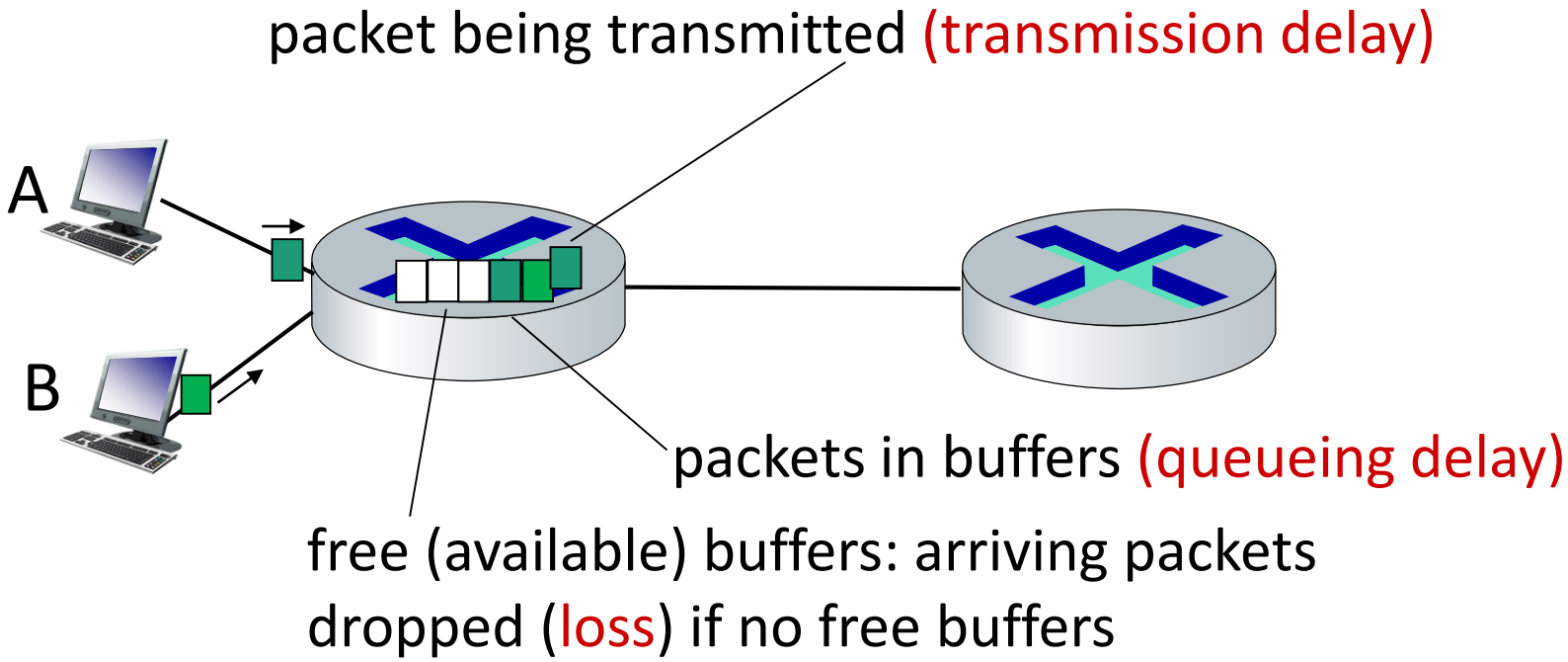
- *circuit-switching:* 10 users
- *packet switching:* with 35 users, probability > 10 active at same time is less than .0004

Chapter 1: roadmap

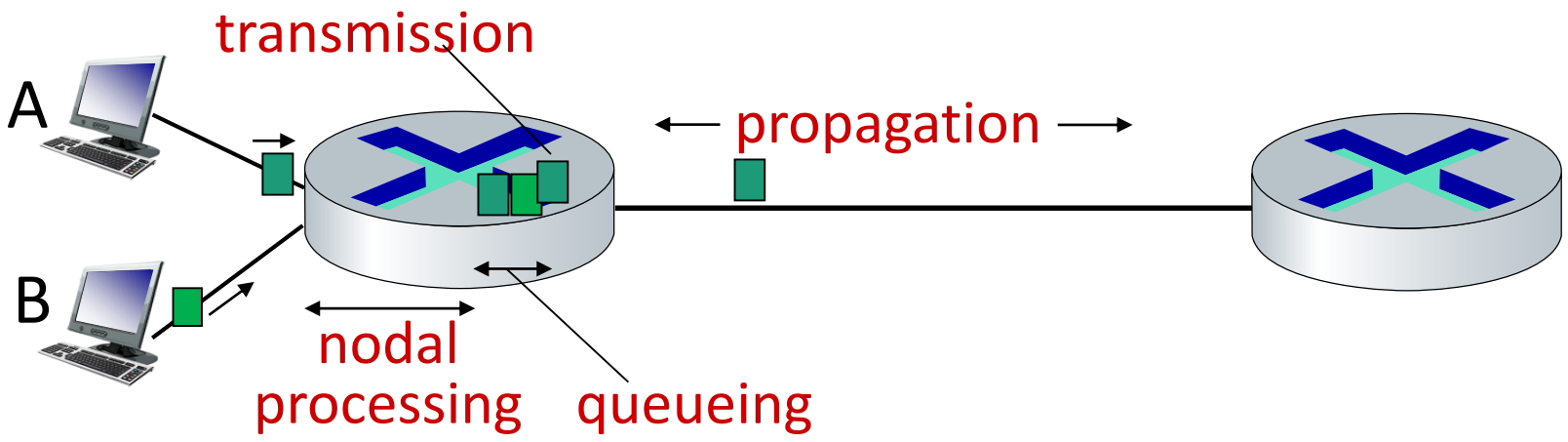
- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models
- History

How do packet delay and loss occur?

- packets *queue* in router buffers, waiting for turn for transmission
  - queue length grows when arrival rate to link (temporarily) exceeds output link capacity
- packet *loss* occurs when memory to hold queued packets fills up



Packet delay: four sources



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

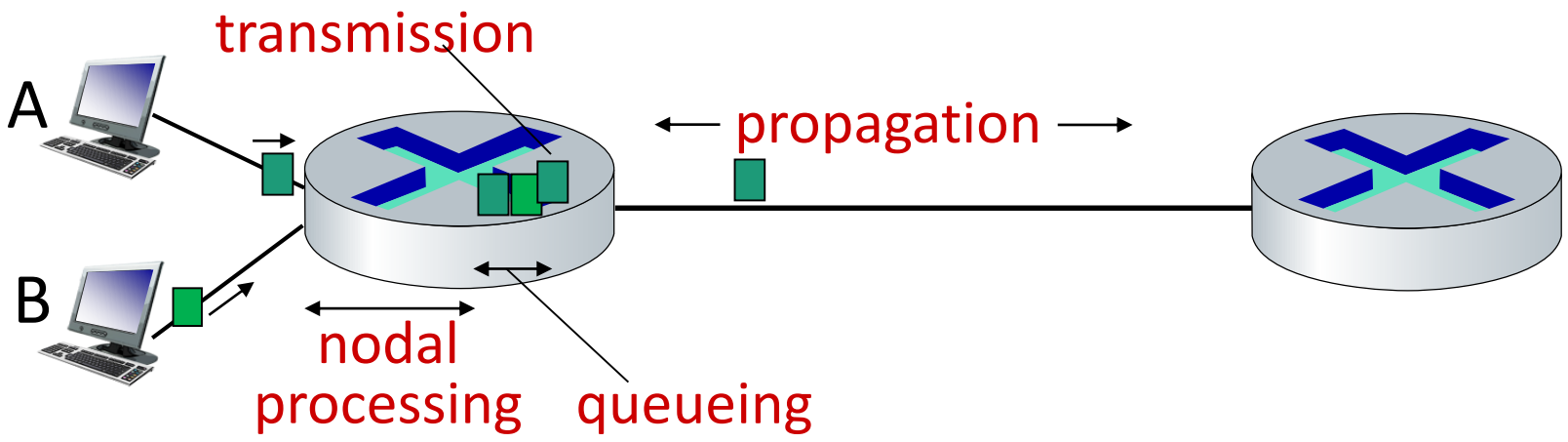
d_{proc}: nodal processing

- check bit errors
- determine output link
- typically < microsecs

d_{queue}: queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Packet delay: four sources



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{trans}: transmission delay:

- L: packet length (bits)
- R: link *transmission rate (bps)*
- $d_{trans} = L/R$

d_{prop}: propagation delay:

- d: length of physical link
- s: propagation speed (~2×10^8 m/sec)
- $d_{prop} = d/s$

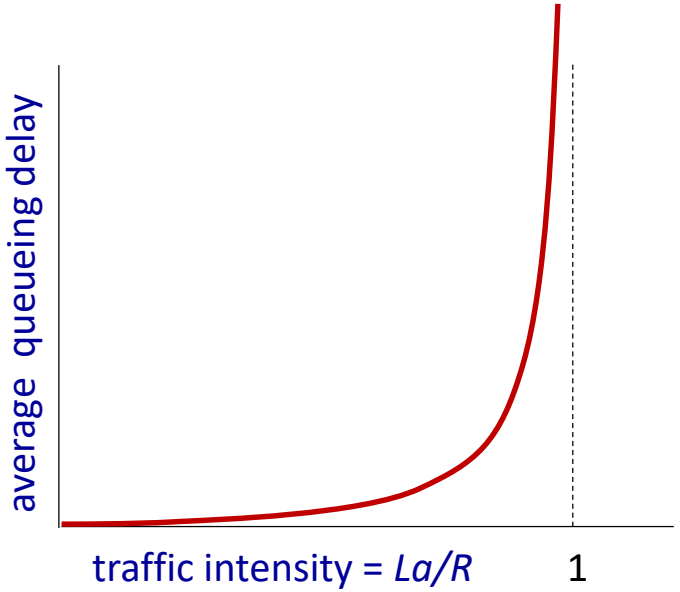
d_{trans} and d_{prop} *very* different

Packet queueing delay (revisited)

- *a:* average packet arrival rate
- *L:* packet length (bits)
- *R:* link bandwidth (bit transmission rate)

$$\frac{L \cdot a}{R} : \frac{\text{arrival rate of bits}}{\text{service rate of bits}}$$

"traffic intensity"



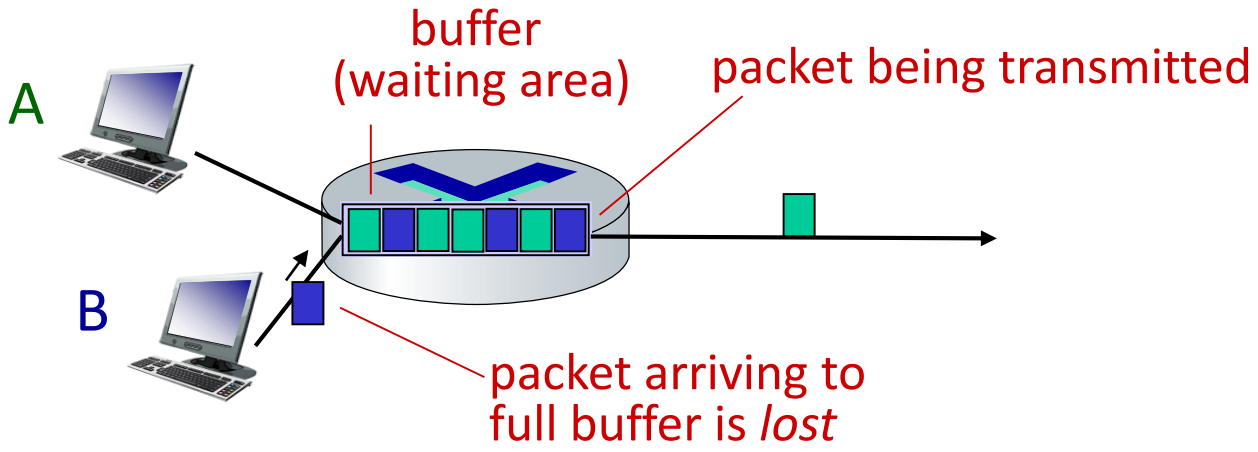
- *La/R* ~ 0: avg. queueing delay small
- *La/R* -> 1: avg. queueing delay large
- *La/R* > 1: more "work" arriving is more than can be serviced - average delay infinite!

La/R ~ 0

La/R -> 1

Packet loss

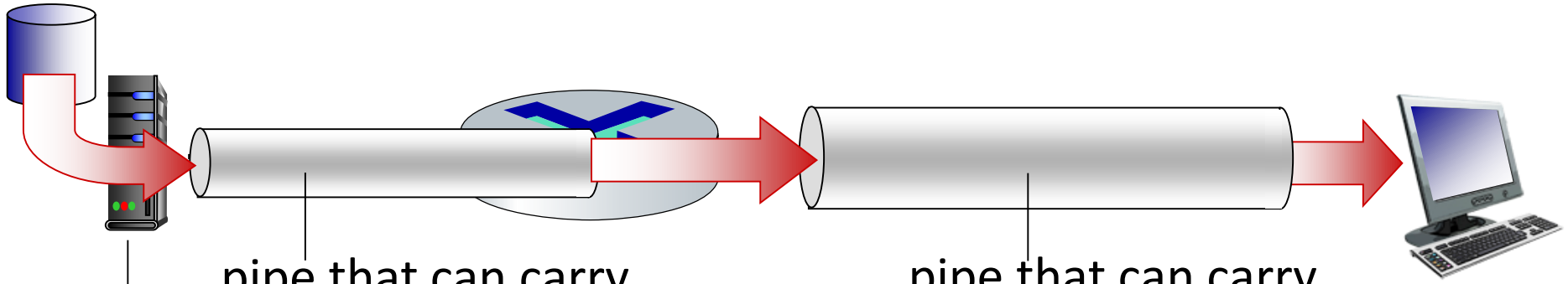
- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all



* Check out the Java applet for an interactive animation (on publisher's website) of queuing and loss

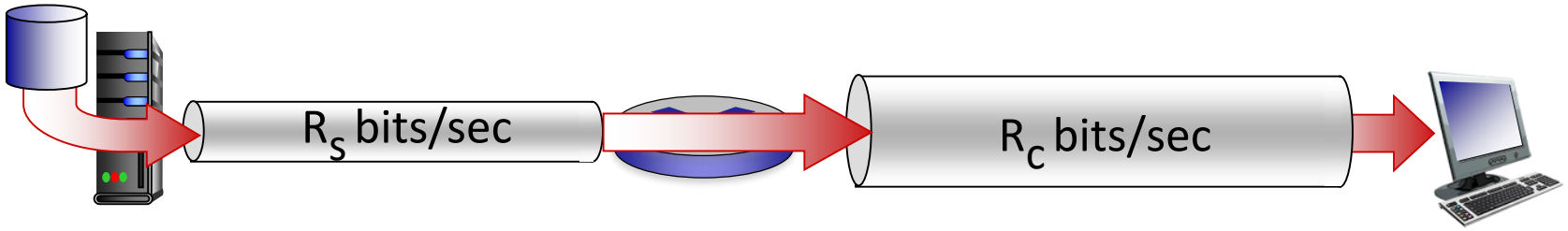
Throughput

- *throughput:* rate (bits/time unit) at which bits are being sent from sender to receiver
  - *instantaneous:* rate at given point in time
  - *average:* rate over longer period of time

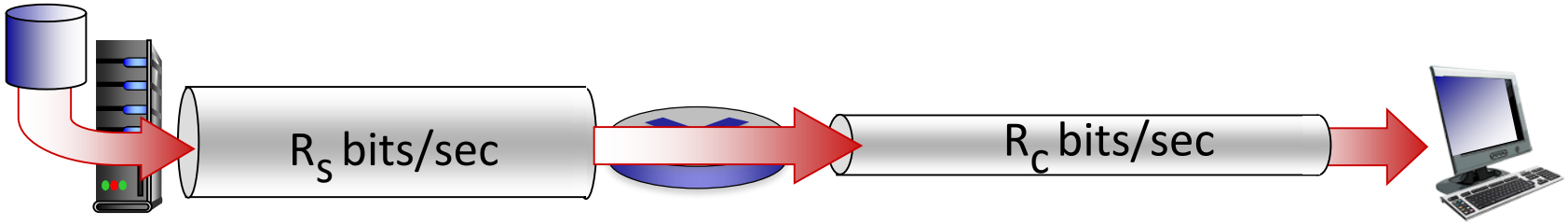


Throughput

$R_s < R_c$ What is average end-end throughput?



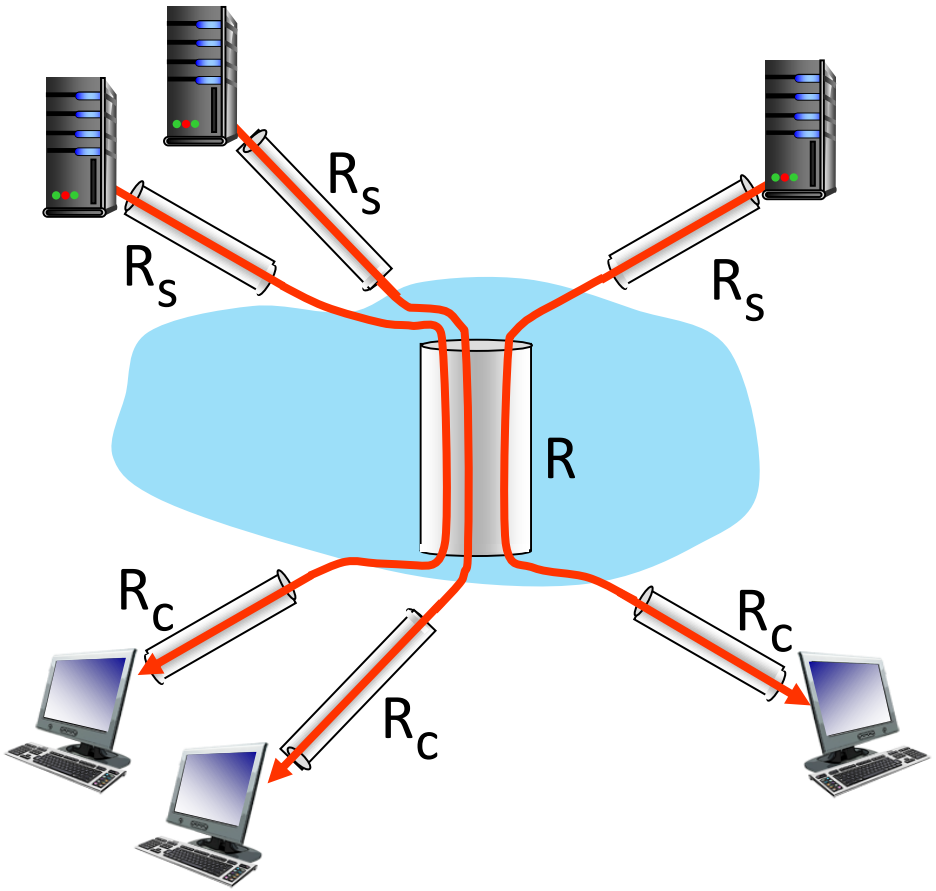
$R_s > R_c$ What is average end-end throughput?



bottleneck link

link on end-end path that constrains end-end throughput

Throughput: network scenario



10 connections (fairly) share backbone bottleneck link *R* bits/sec

- per-connection end-end throughput: $min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/

Chapter 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models
- History

Protocol “layers” and reference models

Networks are complex, with many “pieces”:

- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

Question: is there any hope of *organizing* structure of network?

- and/or our *discussion* of networks?

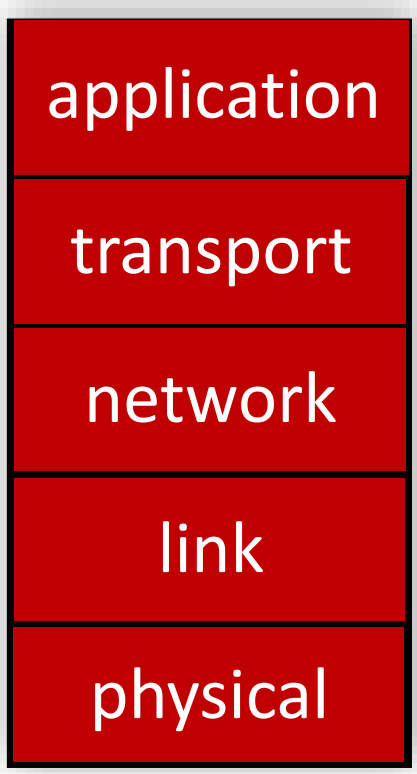
Why layering?

Approach to designing/discussing complex systems:

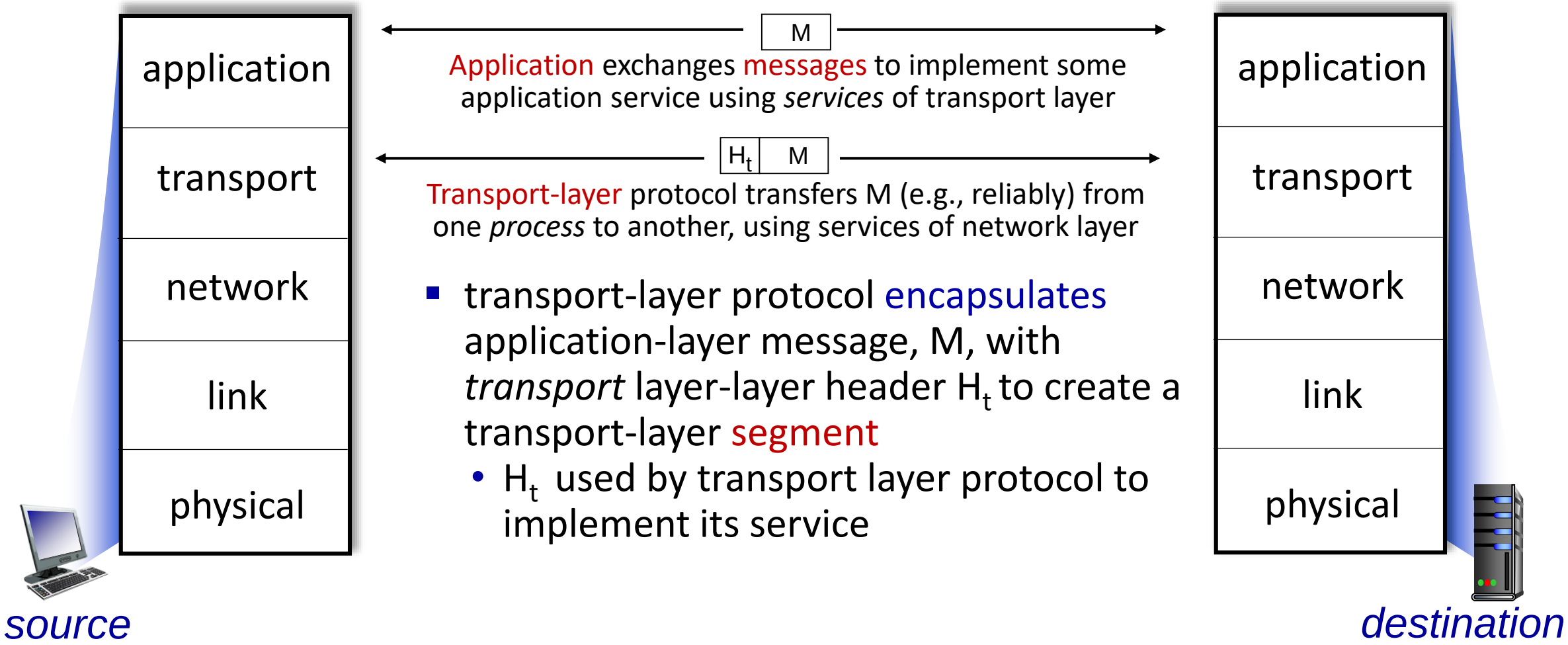
- explicit structure allows identification, relationship of system's pieces
  - layered *reference model* for discussion
- modularization eases maintenance, updating of system
  - change in layer's service *implementation*: transparent to rest of system
  - e.g., change in gate procedure doesn't affect rest of system

Layered Internet protocol stack

- *application:* supporting network applications
  - HTTP, IMAP, SMTP, DNS
- *transport:* process-process data transfer
  - TCP, UDP
- *network:* routing of datagrams from source to destination
  - IP, routing protocols
- *link:* data transfer between neighboring network elements
  - Ethernet, 802.11 (WiFi), PPP
- *physical:* bits "on the wire"

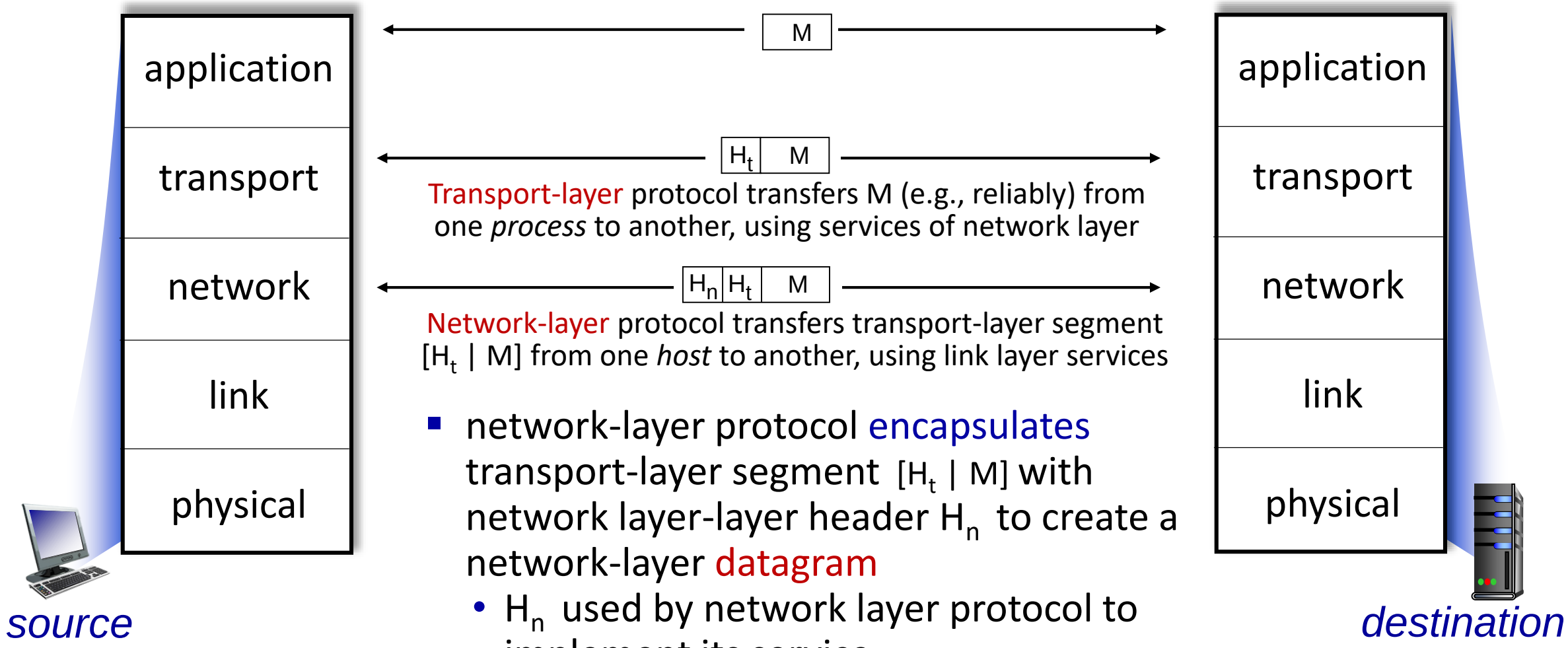


Services, Layering and Encapsulation



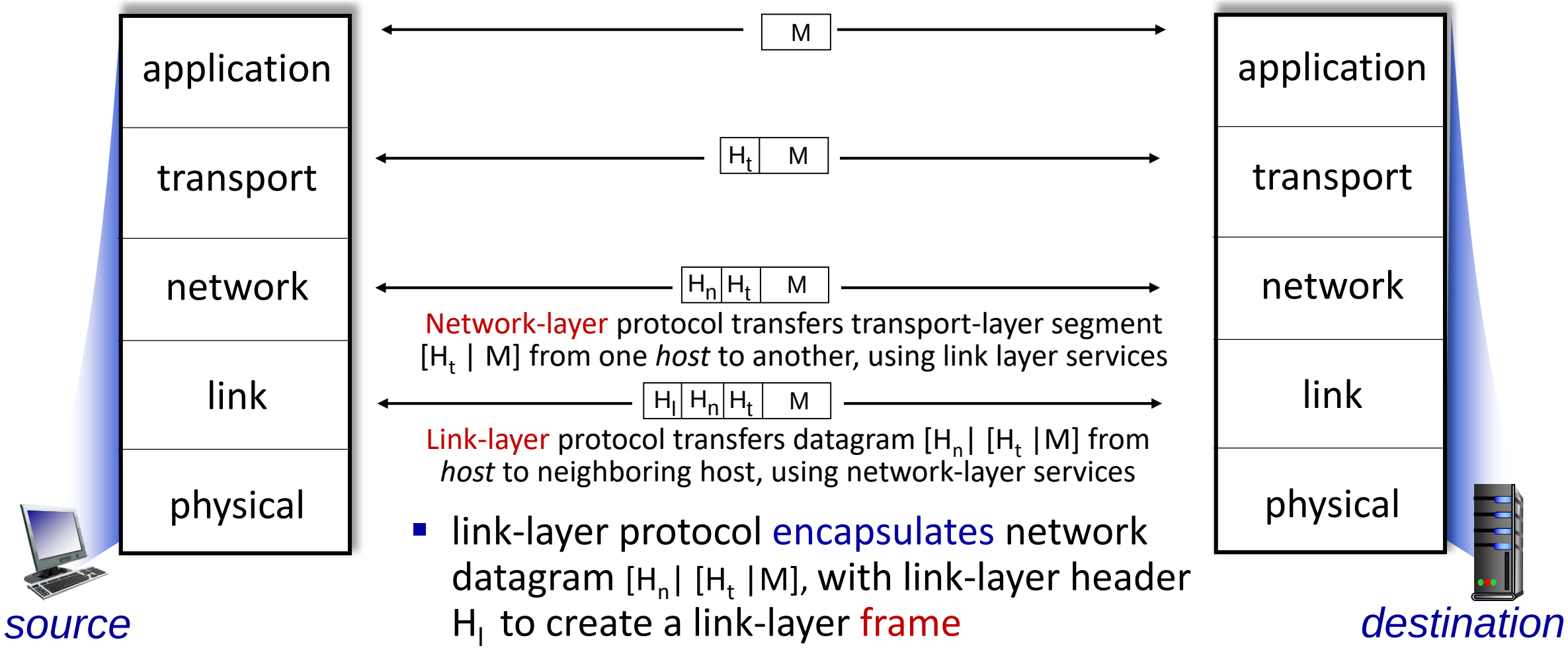
- transport-layer protocol encapsulates application-layer message, M, with *transport* layer-layer header H_t to create a transport-layer segment
  - H_t used by transport layer protocol to implement its service

Services, Layering and Encapsulation



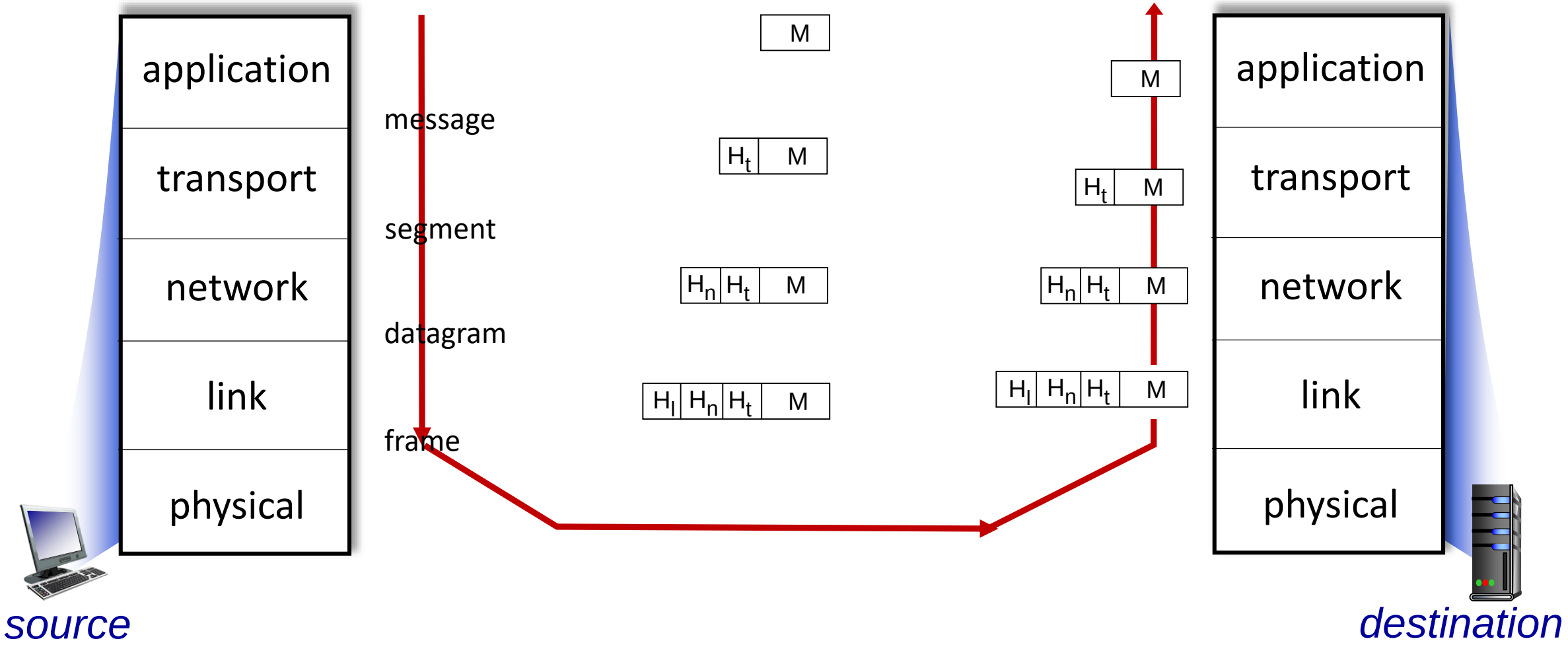
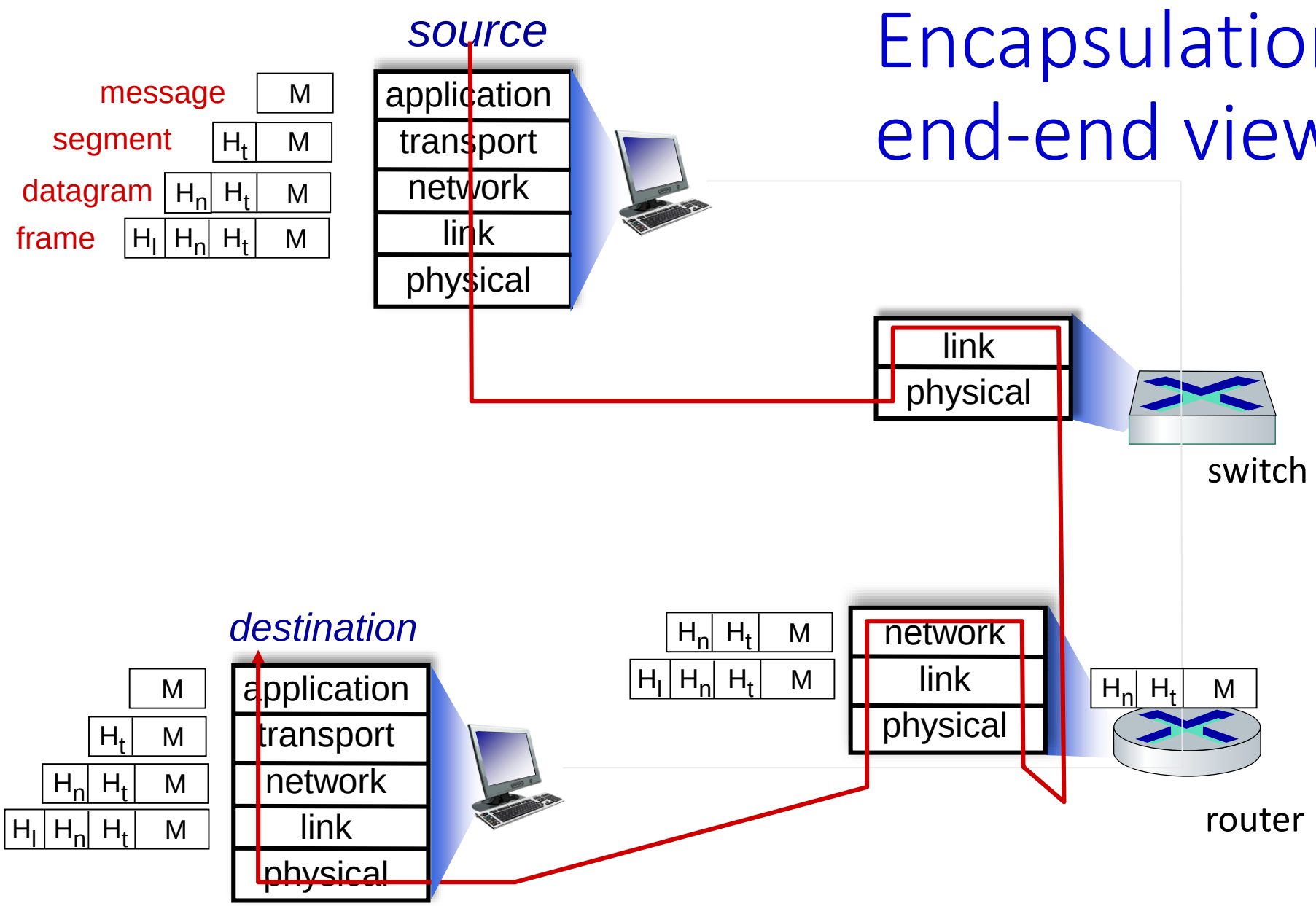
- network-layer protocol encapsulates transport-layer segment [H_t | M] with network layer-layer header H_n to create a network-layer datagram
  - H_n used by network layer protocol to implement its service

Services, Layering and Encapsulation



- link-layer protocol encapsulates network datagram [H_n| [H_t |M], with link-layer header H_l to create a link-layer frame

Services, Layering and Encapsulation



Encapsulation: an end-end view